

Bulkhead Pattern

1. Overview

The **Bulkhead Pattern** is a fault-tolerance design pattern where a system is divided into **isolated resource pools** so that failure, overload, or exhaustion in one pool does **not bring down the entire system**.

The term comes from ships:

- Ships are divided into sealed compartments called **bulkheads**
- If one compartment floods, the whole ship doesn't sink

Same idea in software:

None

One failing part should not destroy the full system

2. Why Bulkhead Is Needed

In shared systems, all users/endpoints often use same:

- Thread pool
- Connection pool
- CPU resources
- Worker queue
- Hosts
- Containers
- DB connections

If one workload consumes everything:

None

One bad endpoint / tenant / client

↓

Uses all resources

↓

Entire service becomes unavailable

Bulkhead prevents this.

3. Core Idea

Divide resources into isolated pools.

Examples:

- Separate thread pools per endpoint
- Separate hosts per client
- Separate DB pools per workload
- Separate queues per job type
- Separate pods per priority tier

Then:

None

Pool A failure $\neq$ Pool B failure

4. Basic Example

Without Bulkhead:

None

All requests $\rightarrow$ Shared thread pool (100 threads)

Heavy endpoint uses 100 threads

↓

Other endpoints blocked

With Bulkhead:

None

Search API → 40 threads

Payment API → 30 threads

Profile API → 30 threads

If Search overloads:

None

Payment + Profile still work

5. Example 1: Separate Thread Pools Per Endpoint

Endpoints:

- /search
- /payment
- /login

If /search gets traffic spike:

Without bulkhead:

None

All worker threads busy

Login fails

Payments fail

With bulkhead:

None

/search pool exhausted only

/login and /payment continue

6. Example 2: Host Pools Per Requestor / Tenant

Suppose Service0 serves:

- Service1
- Service2

Without isolation:

None

All traffic load balanced to same host group

If Service1 floods traffic:

None

Service2 starves

With bulkhead:

None

Pool A hosts reserved for Service1

Pool B hosts reserved for Service2

Now Service1 cannot consume all capacity.

7. Example 3: Crashing Request Bug

Suppose a specific request triggers memory crash.

Without bulkhead:

None

Buggy request hits host1 → crash

Retry hits host2 → crash

Retry hits host3 → crash

Entire fleet dies

With bulkhead:

None

Buggy traffic routed only to Pool A

Pool B remains healthy

System partially available.

8. Resource Types Commonly Bulkheaded

Compute Isolation

- CPU quotas
- Containers
- Pods
- VMs

Concurrency Isolation

- Thread pools
- Worker pools

Network Isolation

- Connection pools
- API rate partitions

Storage Isolation

- DB connection pools
- Cache quotas

9. Architecture Example

Without Bulkhead

None

Users A,B,C



Shared Load Balancer



Common Resource Pool

Failure affects everyone.

With Bulkhead

None

User A → Pool A

User B → Pool B

User C → Pool C

Failure isolated.

10. Bulkhead in Microservices

Common in microservices systems:

- Separate outbound connection pools per dependency
- Separate thread pools for blocking calls
- Separate queues for background jobs

Example:

None

Payment service calls:

Bank API

Fraud API

Email API

Each gets separate pool.

If Email API hangs, fraud checks still work.

11. Bulkhead + Circuit Breaker

Strong combination.

None

Dependency Pool Saturated

↓

Circuit Breaker Opens

↓

Reject only that traffic

Other pools unaffected.

12. Prioritization Use Case

Allocate more resources to premium users.

Example:

None

Premium Customers → 70% capacity

Free Tier → 30% capacity

Guarantees SLA for high-priority customers.

13. Trade-off: Idle Capacity

If Pool A idle and Pool B overloaded:

None

Pool A resources unused

Pool B suffering

This is intentional isolation.

Need autoscaling or dynamic rebalancing.

14. Manual vs Dynamic Bulkheads

Static Pools

Fixed capacity.

Easy but rigid.

Dynamic Pools

Auto-adjust pools based on demand.

More efficient but more complex.

15. Real-World Examples

Netflix Hystrix (historically)

Separate thread pools for dependencies.

Kubernetes

Namespaces, quotas, separate deployments.

Cloud Multi-Tenant SaaS

Dedicated pods per enterprise customer.

Banking

Separate resources for card payments vs transfers.

16. Monitoring Required

Bulkheads need visibility.

Track:

- Pool utilization
- Queue depth
- Rejections
- Error rate
- Latency per pool
- Saturation %

If Pool A exhausted often, scale it.

17. Interview Answer

Q: How do you stop one client or endpoint from taking down the entire service?

Answer:

I'd use the Bulkhead pattern by isolating resources into pools. For example, separate thread pools per endpoint or dedicated host pools per tenant. If one pool is overloaded or fails, other pools continue serving traffic.

18. Where to Apply in HLD

Use bulkhead when system has:

- Multiple tenants
- Critical vs noncritical APIs

- Untrusted third-party integrations
- Different traffic patterns
- Risk of noisy neighbors
- Need for partial availability

19. Best Practices

- Start with logical isolation
- Monitor pool saturation
- Combine with autoscaling
- Use circuit breakers
- Keep fallback responses ready
- Separate premium workloads
- Revisit allocations regularly

20. Short Revision Notes

None

Bulkhead = Divide resources into isolated pools

Examples:

- Thread pools per API
- Hosts per tenant
- Queues per workload

Goal:

Failure in one pool should not impact others

Tradeoff:

Isolation reduces sharing efficiency